Debugging

Use of the debugger Statement

1. **Setting Up**:
    - Place the debugger statement before calling a function in the code. For example:

javascript

Copy code

```
console.log("before outer() call");
debugger;
console.log(outer());
console.log("after outer() call");
```

    - Save the changes and reload the browser page.
2. **Execution Behavior**:
    - The program halts at the debugger statement.
    - The browser switches to the **Debugger** tab (Sources in Chrome or Debugger in Firefox).
    - The code line where execution paused is highlighted.
3. **Using the Debugger Console**:
    - Press Esc to toggle the visibility of the console in the Debugger tab.
    - Observe the output for completed statements:

scss

Copy code

```
before outer() call
```

4. **Resume Execution**:
    - Use the **Resume** button (triangle icon or F8) to continue execution.
    - Breakpoints can be added by clicking on line numbers.
    - Example Output:

scss

Copy code

```
before outer() call
```

Hello!

after outer() call

Breakpoints and Execution Control

- **Without the debugger Statement**:
    - Remove the debugger line from the code.
    - Use breakpoints in Developer Tools on specific lines (e.g., console.log calls).
    - Reload the page to observe program behavior.
- **Step-by-Step Execution**:
    - **Step Over (F10)**: Executes the next line without entering a called function.
    - **Step Into (F11)**: Enters a function call for line-by-line execution inside it.
    - **Step Out**: Exits the current function and returns to the caller.

Analyzing Function Call Stack

- The **Call Stack** panel displays the hierarchy of active functions.
- Example:
    - Inside inner function, the stack might show:

sql

Copy code

inner

outer

(anonymous)

- Switch between contexts by clicking on function names in the Call Stack.

Viewing and Modifying Variables

- Use console.log or the **Watch** panel to monitor variable values.
- Modify variables during execution:

javascript

Copy code

```
console.log(name); // -> inner

name = "new name";

console.log(name); // -> new name
```

Measuring Code Execution Time

1. **Basic Example**:
    - Use console.time and console.timeEnd for precise timing:

javascript

Copy code

```
console.time('Task');
// Code to measure
console.timeEnd('Task'); // -> Task: <time> ms
```

2. **Practical Use Case**:
    - Calculating π using the Leibniz formula:

javascript

Copy code

```
console.time('Leibniz');
let part = 0;
for (let k = 0; k < 10000000; k++) {
   part += ((-1) ** k) / (2 * k + 1);
}
console.timeEnd('Leibniz'); // -> Leibniz: <time> ms
let pi = part * 4;
console.log(pi);
```

3. **Optimizing Execution**:
    - Replace complex operations with simpler alternatives where possible.
    - For example, substitute (-1) ** k with (k % 2 === 0 ? 1 : -1) to avoid expensive exponentiation.